Experimental setup: environment, API, and configuration

Experimental setup

Environment

Requirements: Python 3.11 or newer (enforced in `pyproject.toml`), plus an optional Rust toolchain (`cargo`, stable 1.70+) when building native acceleration crates under `../cogant/rust/`.

From the COGANT package root `../cogant/` (where `pyproject.toml` and `py/cogant/` live), install with `uv sync --all-extras`, or `pip install -e ".[dev,viz]"` / `pip install -e ".[all]"` as in GETTING_STARTED.md. Run those commands from that directory when working inside this monorepo layout. Python sources live under `../cogant/py/cogant/`; see the package README.md.

Running the API

Minimal **Session** run:

```
from cogant import Session

session = Session.from_target("./path/to/repo")
session.extract_static()
session.build_graph()
session.translate_to_gnn()
session.export_all("output/")
```

Minimal **Pipeline** run:

```
from cogant import PipelineRunner
from cogant.api.pipeline import PipelineConfig

runner = PipelineRunner()
config = PipelineConfig(output_dir="output/")
bundle = runner.run("./path/to/repo", config)
```

Adjust `PipelineConfig.stages`, `skip_stages`, and `plugins` to

Configuration files

YAML configuration can drive pipeline behavior (paths, stages, plugin options). `../cogant/docs/architecture/README.md` and `../cogant/docs/reference/implementation_status.md` describe the configuration surface; keep project-specific secrets out of version control.

A minimal pipeline configuration looks like this:

```yaml
# cogant.config.yaml
pipeline:
  stages:
    - ingest
    - static
    - normalize
    - graph
    - dynamic        # optional; skipped if no coverage/tra
    - translate
    - statespace
    - process
```

CLI

Use the `cogant` CLI for scripted batch runs—see `../cogant/docs/cli/README.md` for the command list that matches the installed version.